FinCore

Financial Transaction Processing, Ledger & Reconciliation Platform

Project Type: Backend Engineering / FinTech Infrastructure
Project Architecture: Modular Monolith, with the ability to introduce asynchronous workers and event-driven components
Primary Domain: Digital Payments, Financial Ledger Management, Transaction Reconciliation
Project Status: Proposed
Primary Technology: C# and ASP.NET Core Web API
Database: SQL Server or PostgreSQL
Deployment Target: Dockerized application with automated testing and CI/CD

---

1. Executive Summary

FinCore is a backend-focused financial transaction processing platform designed to simulate the core infrastructure behind modern digital payment systems.

The platform provides a structured environment for managing payment requests, tracking transaction lifecycles, maintaining an auditable financial ledger, processing asynchronous events, handling payment-provider notifications, and reconciling internal transaction records against external settlement data.

Unlike a conventional payment CRUD application, FinCore focuses on the engineering challenges that arise when financial operations must remain correct, consistent, traceable, and recoverable under failures, duplicate requests, concurrent execution, and unreliable external integrations.

The system is designed around financial correctness and operational reliability. It emphasizes transaction boundaries, idempotent request processing, immutable financial records, controlled state transitions, reliable background processing, and automated reconciliation.

FinCore will operate as a **simulation platform** using mock payment providers and synthetic settlement data. It will not process real money or store real payment credentials.

The primary objective is to design and implement a production-oriented backend system that demonstrates a strong understanding of financial domain modeling, database consistency, API

design, asynchronous processing, distributed systems concepts, testing, and operational observability.

2. Problem Statement

Financial transaction systems face challenges that go beyond accepting a request and updating a database record.

A payment request may be submitted multiple times because a client retries after a timeout. An external provider may successfully process a payment while the application fails to receive its response. A webhook may arrive more than once, arrive late, or arrive out of order. Internal transaction records may also differ from the settlement information supplied by an external provider.

If these situations are not handled carefully, a system can create duplicate financial effects, report incorrect payment statuses, produce inconsistent ledger entries, or fail to identify missing and mismatched transactions.

FinCore addresses these challenges by establishing explicit rules for transaction processing, state transitions, ledger posting, event handling, and reconciliation.

The system must not assume that a failed HTTP request necessarily means that the financial operation failed. It must distinguish between a confirmed failure and an unknown outcome, then provide a safe mechanism for resolving that uncertainty.

3. Project Objectives

The project aims to:

1. Design a maintainable backend architecture using ASP.NET Core and domain-oriented modules.
2. Implement a payment lifecycle with explicit, validated state transitions.
3. Prevent duplicate financial effects through idempotency and database-level safeguards.
4. Maintain an auditable, double-entry ledger that preserves the history of posted financial records.
5. Process provider notifications and background tasks reliably.
6. Reconcile internal transactions with external settlement records.
7. Detect and manage discrepancies, duplicate records, missing transactions, and amount mismatches.
8. Demonstrate safe database transaction handling and concurrency control.
9. Introduce asynchronous messaging, retry policies, and failure recovery where appropriate.

10. Establish automated tests, structured logging, metrics, tracing, containerization, and CI/CD practices.

4. Scope and System Boundaries

FinCore will simulate a payment processing environment rather than act as a real payment gateway.

The application will include:

- A payment management API.
- A simulated payment-provider integration.
- A financial ledger module.
- A webhook receiving and processing mechanism.
- Background workers for asynchronous operations.
- A reconciliation engine.
- Administrative endpoints for investigating transactions and discrepancies.
- Authentication, authorization, audit trails, and operational monitoring.

The system will not initially include real bank connectivity, actual fund transfers, card-data storage, regulatory certification, or production payment-provider credentials.

These boundaries keep the project achievable while preserving the important engineering problems found in financial backend systems.

5. Core Functional Modules

5.1. Payment Management

The Payment Management module is responsible for creating, retrieving, and tracking payment operations.

Main capabilities

- Create payment requests.
- Retrieve payment details by identifier.
- List and filter payments by status, date, amount, and reference.
- Retrieve payment processing history.
- Record provider references and processing outcomes.
- Support cancellation requests when permitted by the payment state and business rules.
- Expose payment status and relevant failure information.

Payment lifecycle

A simplified lifecycle may include:

- `Pending`
- `Processing`
- `Succeeded`
- `Failed`
- `Cancelled`

The implementation must define the allowed transitions between states. For example, a terminal state must not be overwritten by an arbitrary retry, and a successful payment must not silently become failed because a later request timed out.

The lifecycle should be implemented as an explicit domain rule rather than relying on unrestricted status updates through an API.

The exact lifecycle and transition rules will be documented before implementation.

5.2. Idempotency and Duplicate Request Protection

Idempotency is one of the most important engineering requirements in FinCore.

Clients frequently retry requests because of network failures, timeouts, or uncertain responses. The platform must ensure that repeating the same logical request does not create multiple payment operations or duplicate ledger effects.

Main capabilities

- Accept an idempotency key for payment-creation requests.
- Associate the key with the relevant client or account scope.
- Persist the result and processing state associated with the key.
- Return the original result when the same valid request is retried.
- Detect reuse of the same key with a different request payload.
- Enforce uniqueness at the database level.
- Handle concurrent requests using the same key safely.
- Define the retention and expiration policy for idempotency records.

Idempotency must not depend solely on an in-memory check. The implementation must remain correct when multiple requests execute concurrently or when the application restarts.

The system must also distinguish a retry of an existing payment from a request to create a genuinely new payment.

5.3. Financial Ledger

The Ledger module records the financial effects of completed operations.

Instead of treating a payment status or a mutable balance field as the complete financial record, FinCore will maintain ledger transactions containing balanced debit and credit entries.

Core principles

- Every posted journal transaction must balance.
- Ledger entries must be recorded atomically.
- Posted financial entries must be immutable.
- Corrections should be represented through reversing or compensating entries rather than silently rewriting historical records.
- Every entry must be traceable to its originating business operation.
- Duplicate event processing must not create duplicate financial postings.
- Financial amounts must use appropriate decimal precision and currency-aware validation.

Illustrative example

For a simulated transfer of 100.00 units between two internal accounts, the ledger may record:

Account	Debit	Credit
Source account	100.00	0.00
Destination account	0.00	100.00
Total	**100.00**	**100.00**

This example illustrates a balanced internal transfer. The actual journal accounts and entries for payment authorization, settlement, fees, refunds, and provider clearing must be defined according to the selected financial model.

FinCore must distinguish between a payment's operational status and its financial posting status. A payment being marked successful must not, by itself, guarantee that a correct ledger posting has occurred unless the relevant operation has completed under the defined transaction rules.

Ledger invariants

- Total debits must equal total credits for every posted journal transaction.
- Journal entries must reference valid accounts.
- A journal transaction must not be partially posted.

- A business operation must not produce duplicate financial postings.
- Account and journal history must remain auditable.

5.4. Payment Provider Simulator

The Provider Simulator represents an external payment service without requiring a real integration.

It will simulate realistic provider behavior, including successful responses, declined payments, delayed responses, timeouts, duplicate notifications, and temporary failures.

Main capabilities

- Generate provider transaction references.
- Simulate successful and unsuccessful payment outcomes.
- Simulate uncertain outcomes in which the provider may have processed the operation but the application did not receive confirmation.
- Generate provider webhook events.
- Support deterministic scenarios for testing failure recovery.
- Provide a controlled way to query or reconcile the provider's simulated transaction state.

The simulator will allow FinCore to demonstrate how a backend system handles unreliable external dependencies without risking real financial transactions.

5.5. Webhook Processing

The Webhook module receives notifications from the simulated provider.

Because external systems may deliver the same event multiple times, webhook handling must be designed around duplicate delivery and safe processing.

Main capabilities

- Validate the simulated provider's webhook authenticity mechanism.
- Persist incoming event metadata.
- Identify events using a stable provider event identifier.
- Reject or safely acknowledge duplicate deliveries.
- Process events asynchronously when appropriate.
- Track processing status, attempts, and failure reasons.
- Retry transient failures under a defined policy.
- Prevent a duplicate event from causing repeated ledger postings.
- Handle late and out-of-order events according to explicit business rules.

Where appropriate, the system will separate webhook receipt from business processing. A successfully received event should be durably recorded before the application acknowledges it, so that a later worker failure does not cause the event to disappear.

5.6. Reconciliation Engine

The Reconciliation Engine compares FinCore's internal payment and settlement records with an external provider's settlement data.

It is a central business capability of the project, rather than a simple report-generation feature.

Input sources

- Internal payment records.
- Internal ledger and settlement records.
- Simulated provider transaction records.
- Imported settlement files, such as CSV files.

Reconciliation outcomes

The engine should identify cases such as:

- **Matched:** Internal and provider records agree under the configured matching rules.
- **Missing internally:** A provider settlement record has no corresponding internal record.
- **Missing externally:** An internal transaction expected in the settlement data is absent.
- **Amount mismatch:** Matching references exist, but the financial amounts differ.
- **Duplicate provider record:** The imported data contains repeated provider transactions.
- **Status mismatch:** The internal state conflicts with the provider's reported state.
- **Unresolved:** The available information is insufficient to determine a reliable match.

Main capabilities

- Import and validate settlement files.
- Track each import batch and its processing status.
- Normalize external records before matching.
- Match records using provider references and defined fallback rules.
- Detect duplicate and unmatched records.
- Compare amounts, currencies, statuses, and settlement dates where applicable.
- Persist reconciliation results and exception details.
- Support safe reprocessing of an import batch.
- Provide endpoints for investigating unresolved discrepancies.
- Maintain an audit trail of reconciliation decisions.

Reconciliation must be repeatable. Reprocessing the same file should not silently duplicate settlement records or create repeated financial effects.

A discrepancy should be reported and investigated rather than automatically forcing internal data to match external data.

5.7. Background Processing

Some operations should not execute entirely within the lifetime of an HTTP request.

FinCore will use background processing for tasks such as:

- Processing queued provider events.
- Retrying eligible failed operations.
- Executing scheduled reconciliation jobs.
- Processing imported settlement batches.
- Detecting operations that remain in an uncertain or incomplete state.
- Generating operational summaries.

The implementation may begin with ASP.NET Core hosted services and a durable database-backed work queue where appropriate. RabbitMQ and dedicated workers can be introduced in a later phase.

The system must define retry limits, retry delays, terminal failure behavior, and recovery procedures. Retries must be safe for the underlying business operation.

5.8. Authentication, Authorization, and Audit

FinCore will expose protected endpoints for operational and administrative actions.

Proposed roles

- **Administrator:** Manage system configuration and authorized administrative operations.
- **Operations Analyst:** View transactions, inspect reconciliation results, and investigate discrepancies.
- **Service Client:** Submit and retrieve payments within its authorized scope.

These roles are a starting point and should be refined as the domain model develops.

Security requirements

- Token-based authentication using JWT or an appropriate ASP.NET Core Identity configuration.
- Role-based and, where necessary, resource-level authorization.

- Request validation and consistent error responses.
- Rate limiting for sensitive endpoints.
- Protection of secrets and configuration values.
- Input validation for imported files and external event payloads.
- Audit logging for sensitive administrative operations.
- No storage of real card security codes or unnecessary sensitive payment credentials.

Audit records should capture relevant actors, timestamps, actions, and affected resources without exposing secrets or sensitive authentication material.

6. Proposed Architecture

FinCore will begin as a **Modular Monolith**.

This approach keeps the deployment and development model manageable while establishing clear boundaries between business capabilities. Messaging and separate worker processes can be introduced later without requiring the entire system to be redesigned.

Logical components

1. **API Layer:** HTTP endpoints, authentication, request validation, and response handling.
2. **Application Layer:** Use cases, orchestration, authorization checks, and transaction coordination.
3. **Domain Layer:** Payment lifecycle rules, ledger invariants, reconciliation rules, and domain models.
4. **Infrastructure Layer:** Database access, provider simulation, message broker integration, background execution, and external adapters.
5. **Operational Layer:** Structured logging, metrics, distributed tracing, health checks, and deployment configuration.

Proposed modules

- Payments
- Ledger
- Provider Integration
- Webhooks
- Reconciliation
- Background Processing
- Identity and Access
- Audit and Observability

Modules should communicate through clearly defined interfaces and application-level contracts. Domain rules should not depend directly on HTTP controllers, database-specific APIs, or message-broker implementation details.

6.1. Data Consistency and Transaction Boundaries

FinCore must explicitly define which changes belong to the same database transaction.

For example, when a business operation requires both a state change and a ledger posting, the implementation must prevent a partial result in which only one of those changes is committed.

For asynchronous communication, the project may introduce the transactional outbox pattern. The outbox allows business data and an event-to-be-published record to be persisted within the same database transaction. A background publisher then delivers the event to the message broker.

The outbox does not guarantee that a broker will receive an event exactly once. Consumers must still be idempotent and tolerate duplicate delivery.

The design must avoid assuming that a database transaction can atomically include an external payment-provider operation or a RabbitMQ publish without an additional coordination strategy.

7. Proposed Technology Stack

The final stack should be selected after evaluating the project's requirements and implementation complexity.

Area	Proposed technology
Programming language	C#
Backend framework	ASP.NET Core Web API
ORM	Entity Framework Core
Relational database	SQL Server or PostgreSQL
Authentication	ASP.NET Core Identity and JWT, where appropriate
Unit testing	xUnit
API and integration testing	ASP.NET Core integration testing tools

Background processing	Hosted Services initially; a durable job mechanism as needed
Messaging	RabbitMQ in a later phase
Caching	Redis where justified by measured needs
API documentation	OpenAPI / Swagger
Logging	Structured logging with Serilog or an equivalent provider
Observability	OpenTelemetry
Containers	Docker and Docker Compose
CI/CD	An automated build, test, and deployment pipeline

The initial implementation should use one relational database as the source of truth for payment and ledger records. Redis should not be treated as the authoritative store for financial balances or ledger entries.

8. Non-Functional Requirements

8.1. Correctness and Consistency

- Prevent duplicate business operations where idempotency is required.
- Preserve ledger balance invariants.
- Define valid state transitions.
- Use appropriate database constraints and transaction isolation.
- Protect against concurrent requests and race conditions.
- Preserve an auditable history of posted financial records.

8.2. Reliability and Recovery

- Recover from application restarts without losing durably accepted work.
- Retry transient failures according to a controlled policy.
- Prevent repeated financial effects during retries.
- Track unresolved payment outcomes.
- Provide mechanisms to inspect and recover failed background operations.

8.3. Security

- Authenticate and authorize protected operations.
- Validate external events and imported files.
- Protect secrets and sensitive information.
- Record relevant administrative actions.
- Avoid leaking internal exception details through public API responses.

8.4. Maintainability

- Enforce clear module boundaries.
- Separate domain rules from infrastructure concerns.
- Use consistent error handling and API response conventions.
- Maintain automated tests for critical business invariants.
- Document important architectural decisions.

8.5. Observability

- Emit structured logs with relevant transaction and correlation identifiers.
- Expose health checks for application dependencies.
- Record useful metrics for payment outcomes, processing failures, queue backlogs, and reconciliation discrepancies.
- Use distributed tracing when requests cross asynchronous processing boundaries.
- Avoid placing sensitive payment data in logs or trace attributes.

9. Testing Strategy

Testing will focus not only on successful requests but also on financial correctness and failure scenarios.

Unit tests

- Payment state transition rules.
- Idempotency decisions.
- Ledger balancing invariants.
- Reconciliation matching rules.
- Retry eligibility and failure classification.

Integration tests

- Database transactions and rollback behavior.
- Unique constraints and duplicate request handling.
- Concurrent payment requests.

- Atomic ledger posting.
- Webhook persistence and processing.
- Reconciliation batch imports.
- Authorization and API validation.

Failure and reliability tests

- The same payment request is submitted repeatedly.
- Two requests use the same idempotency key concurrently.
- A provider succeeds but the application receives a timeout.
- A webhook is delivered multiple times.
- A worker crashes after receiving a message.
- An event is published more than once.
- An imported settlement file contains duplicate records.
- A reconciliation batch is processed again.
- A database failure occurs during a multi-record financial operation.

The expected result of each scenario must be defined before implementation. For critical scenarios, tests should verify database state and ledger effects, not merely HTTP status codes.

10. Implementation Roadmap

Phase 1 — Foundation and Payment API

- Establish the solution structure.
- Configure ASP.NET Core, EF Core, and the relational database.
- Implement the initial payment model and API.
- Add request validation, pagination, filtering, and consistent error handling.
- Add Swagger and basic unit tests.

Deliverable: A functional payment API with a documented domain model.

Phase 2 — Payment Lifecycle and Idempotency

- Define valid payment states and transitions.
- Implement idempotency keys and request fingerprinting.
- Add database uniqueness constraints.
- Handle duplicate and concurrent requests.
- Define the behavior for unknown provider outcomes.

Deliverable: A payment API that safely handles retries and invalid state transitions.

Phase 3 — Financial Ledger

- Design accounts, journal transactions, and journal entries.
- Enforce balanced debit and credit postings.
- Implement atomic posting and immutability rules.
- Associate ledger postings with originating business operations.
- Test duplicate posting prevention and rollback behavior.

Deliverable: A reliable internal ledger with verifiable financial invariants.

Phase 4 — Provider Simulation

- Implement a simulated provider adapter.
- Generate provider references and outcomes.
- Simulate success, decline, timeout, and uncertain outcomes.
- Implement a controlled mechanism for retrieving provider transaction state.

Deliverable: A reproducible environment for testing external payment failures.

Phase 5 — Webhooks and Reliable Processing

- Receive and persist provider events.
- Implement duplicate-event detection.
- Add asynchronous processing and retry policies.
- Track processing attempts and terminal failures.
- Introduce the transactional outbox where required.

Deliverable: A recoverable event-processing workflow.

Phase 6 — Reconciliation Engine

- Import settlement files.
- Validate and normalize provider records.
- Implement matching and discrepancy detection.
- Persist reconciliation batches and results.
- Add endpoints for investigating exceptions.
- Ensure batch reprocessing is safe.

Deliverable: An auditable reconciliation workflow with actionable discrepancy reports.

Phase 7 — Messaging and Background Workers

- Introduce RabbitMQ where asynchronous communication provides a clear benefit.
- Implement producers and consumers.
- Add idempotent consumers, bounded retries, and dead-letter handling.
- Define message contracts and failure recovery procedures.
- Test duplicate delivery and worker restart scenarios.

Deliverable: An event-driven extension with controlled failure handling.

Phase 8 — Performance and Caching

- Measure database query performance.
- Add indexes and optimize expensive queries.
- Introduce Redis for appropriate read-heavy or temporary data.
- Define cache invalidation rules.
- Verify that financial correctness never depends on stale cached values.

Deliverable: A measured performance improvement without weakening financial consistency.

Phase 9 — Security and Operational Readiness

- Finalize authentication and authorization.
- Add audit logging and rate limiting.
- Introduce structured logging, metrics, and distributed tracing.
- Add health checks and operational dashboards.
- Containerize the application and supporting dependencies.
- Configure CI/CD and automated test execution.

Deliverable: A containerized, testable, observable backend.

Phase 10 — Hardening and Demonstration

- Run end-to-end business workflows.
- Execute failure, retry, and concurrency tests.
- Document architecture and data consistency decisions.
- Prepare API examples and reproducible demonstration scenarios.
- Document limitations, known risks, and future improvements.

Deliverable: A portfolio-ready engineering project supported by evidence of correctness and reliability.

11. Success Criteria

FinCore will be considered successful when it can demonstrate the following properties:

1. Repeating an eligible payment request does not create duplicate payment operations.
2. Concurrent requests using the same idempotency key are handled consistently.
3. Every posted journal transaction satisfies the debit-equals-credit invariant.
4. Failed database transactions do not leave partially committed ledger postings.
5. Duplicate webhooks do not produce duplicate financial effects.
6. Uncertain provider outcomes can be investigated and resolved without blindly repeating the operation.
7. Reconciliation identifies defined classes of missing, duplicate, and mismatched records.
8. Reprocessing a settlement batch does not create unintended duplicate records or postings.
9. Background processing failures are visible and recoverable under documented policies.
10. Critical business invariants are covered by automated tests.
11. The application can be built and run using documented setup instructions.
12. Logs, metrics, and traces provide useful diagnostic information without exposing sensitive data.

These criteria should be demonstrated through automated tests, reproducible scenarios, and documented design decisions rather than claims alone.

12. Key Engineering Challenges

The most important challenges in FinCore are expected to be:

- Designing a payment lifecycle that represents uncertainty rather than treating every timeout as a definitive failure.
- Preventing duplicate financial effects across API retries, webhooks, background workers, and message redelivery.
- Maintaining ledger correctness under concurrent requests and database failures.
- Separating operational payment status from financial posting and settlement status.
- Designing reliable processing across database and message-broker boundaries.
- Building reconciliation rules that identify discrepancies without automatically hiding or overwriting them.
- Making every important financial operation traceable and auditable.

These challenges are the core of the project. Infrastructure technologies should be introduced to support them, not simply to increase the number of tools in the stack.

13. Future Enhancements

After the core system is stable, potential extensions include:

- Refund and reversal workflows.
- Multi-currency transaction modeling.
- Fee calculation and settlement accounting.
- Account balance projections derived from ledger entries.
- Merchant-level transaction reporting.
- Advanced reconciliation rules and exception workflows.
- Provider adapters behind a common integration interface.
- More advanced monitoring, alerting, and operational analytics.
- Performance testing and documented capacity benchmarks.
- An administrative dashboard for transaction and reconciliation investigation.

These are future enhancements, not prerequisites for the first working version.

14. Final Project Vision

FinCore is intended to demonstrate the design and implementation of a reliable financial backend rather than a collection of ordinary CRUD endpoints.

Its engineering value comes from the correctness of its financial model, the resilience of its processing workflows, its ability to recover from uncertain external outcomes, and the traceability of every important financial effect.

By combining payment lifecycle management, idempotency, double-entry accounting, reliable asynchronous processing, and automated reconciliation, FinCore provides a practical environment for exploring the engineering principles that underpin dependable financial transaction systems.

The result will be a portfolio project that demonstrates backend engineering skills in domain modeling, relational data design, transactional consistency, distributed processing, testing, security, and production-oriented operations — while remaining a controlled simulation that never handles real funds.

Project Principle: Financial correctness first, reliable processing second, and architectural complexity only when justified by the requirements.